# Train vs Eval Mode

## Pre-Work Module 03

## ECBS5200 — Practical Deep Learning Engineering

# The problem: same model, two behaviors

A PyTorch model can behave **differently** depending on whether it's in **train mode** or **eval mode**.

```
model.train()   # Training behavior
model.eval()    # Inference behavior
```

Same weights. Same architecture. **Potentially different outputs.**

If you don't switch modes correctly, your metrics can be wrong and your deployed model unreliable.

# What `model.train()` does

Activates training-time behaviors:

1. **Dropout is ON** (when configured with dropout > 0)
   - Randomly sets a fraction of neurons to zero on each forward pass
   - Forces the network to not rely on any single neuron — a form of regularization
   - Each forward pass produces a **different** output, even for the same input
2. **BatchNorm updates running statistics**
   - Tracks mean and variance of each batch during training
   - These running stats are used later during inference

# What `model.eval()` does

Activates inference-time behaviors:

1. **Dropout is OFF**
   - All neurons are active — no randomness
   - Output is **deterministic** for the same input
2. **BatchNorm uses frozen running statistics**
   - Uses the mean/variance accumulated during training
   - Does not update them with new data

```
model.eval()  # "I'm done training, give me consistent predictions"
```

# A real-world nuance: ModernBERT defaults to dropout = 0.0

Our course model, ModernBERT-base, ships with **dropout disabled** ( `p=0.0` ).

That means out of the box, train mode and eval mode produce **identical outputs**.

So why does this module exist? Because:

1. **You may add dropout during fine-tuning** — it's common to set 0.1 or 0.2 as regularization
2. **Other models use dropout by default** — BERT, DeBERTa, GPT-2 all ship with non-zero dropout
3. **It's a professional habit** — always call `model.eval()` for inference, no exceptions

In the notebook, we'll set dropout to 0.1 so you can see the effect firsthand.

# Why forgetting `eval()` can burn you

**During validation (when dropout > 0):**

- Dropout is still randomly zeroing neurons
- Your validation accuracy **fluctuates randomly** between runs
- You can't tell if a change improved the model or if you just got lucky

**During deployment:**

- The same customer complaint gets a **different prediction** each time
- Your model appears non-deterministic to users
- You'll spend hours debugging something that "works sometimes"

**The rule: always call `model.eval()` for inference. No exceptions.**

# The companion: `torch.no_grad()`

`model.eval()` changes **model behavior**. `torch.no_grad()` changes **memory usage**.

```
with torch.no_grad():
    outputs = model(**inputs)
```

| | `model.eval()` | `torch.no_grad()` |
|---|---|---|
| **What it does** | Disables dropout, freezes batchnorm | Disables gradient computation |
| **Why you need it** | Correct predictions | Save memory |
| **Affects output?** | Can change the result | No — same result either way |
| **Forgetting it** | Potentially wrong metrics | OOM errors on large batches |

They do **different things**. You need **both** during inference.

# The pattern you'll use every week

```
# === TRAINING LOOP ===
model.train()
for batch in train_loader:
    outputs = model(**batch)
    loss = outputs.loss
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

# === EVALUATION LOOP ===
model.eval()
with torch.no_grad():
    for batch in eval_loader:
        outputs = model(**batch)
        # compute metrics — no loss.backward()!
```

**Every week. Every experiment. This pattern.**

Train loop: `model.train()` — Eval loop: `model.eval()` + `torch.no_grad()`

# Key takeaways

1. `model.train()` enables dropout and batchnorm updates — use it for training
2. `model.eval()` disables dropout and freezes batchnorm — use it for inference
3. **Not all models have the same defaults** — ModernBERT ships with dropout=0.0, but many don't
4. **Always call** `model.eval()` **anyway** — it's a professional habit, not an optional step
5. `torch.no_grad()` saves memory by skipping gradient tracking — different from `eval()`
6. **The pattern**: train loop = `model.train()`, eval loop = `model.eval()` + `torch.no_grad()`

**Next: try it yourself in the notebook!**